PROYECTO

PachocloSystem

API REST: endpoints y métodos HTTP

Microservicios PachocloSystem y MedicamentosService

Integrantes Angel Antonio Suarez Vera

API REST: endpoints y métodos HTTP

PachocloSystem · Sistema Hospitalario · Entrega del corte 2

1. Arquitectura

El sistema tiene dos microservicios Spring Boot independientes y un frontend Angular. Cada servicio tiene su propia base de datos PostgreSQL y no comparten código.

```
Frontend Angular  ──HTTP + JWT──▶  PachocloSystem  ──HTTP + X-Api-Key──▶  MedicamentosService
  (puerto 4200)                      (puerto 8080)                          (puerto 8081)
                                     Auth, pacientes,                       Inventario, stock
                                     trabajadores, usuarios,                y vencimientos
                                     historial clínico
```

- **Frontend → PachocloSystem:** todas las peticiones van a `/api/**` con el token JWT en la cabecera `Authorization: Bearer`.
- **PachocloSystem → MedicamentosService:** comunicación HTTP real mediante `RestClient` (clase `MedicamentosClient`), autenticada con una clave compartida en la cabecera `X-Api-Key`. PachocloSystem es la única puerta de entrada a los medicamentos: el frontend nunca llama directamente a MedicamentosService.
- Al crear un registro clínico de tipo **medicación** que indica medicamento y cantidad, PachocloSystem llama a `POST /api/medicamentos/{id}/salidas` de MedicamentosService para descontar el stock.
- Si MedicamentosService no está disponible, PachocloSystem responde **503** y solo el módulo de medicamentos se ve afectado.

2. Convenciones HTTP

Método	Uso en la API	Respuesta exitosa
GET	Listar o consultar recursos. Los listados aceptan `?q=` para buscar.	200 OK
POST	Crear un recurso o registrar una operación (entrada/salida de stock, login).	201 Created + cabecera `Location`
PUT	Reemplazar todos los datos editables de un recurso.	200 OK
PATCH	Modificar un solo aspecto del recurso (habitación, rol, estado, contraseña).	200 OK / 204 No Content
DELETE	Eliminar un recurso.	204 No Content

Errores: 400 (datos inválidos), 401 (sin autenticar), 403 (rol sin permiso), 404 (no existe), 409 (conflicto), 503 (servicio de medicamentos no disponible).

3. Endpoints de PachocloSystem (puerto 8080)

3.1 AuthController

Microservicio	Método	Endpoint	Descripción
Autenticación	POST	`/api/auth/login`	Iniciar sesión y obtener el token JWT
Autenticación	GET	`/api/auth/me`	Consultar el usuario autenticado
Autenticación	POST	`/api/auth/password`	Cambiar la contraseña propia

3.2 PacienteController

Microservicio	Método	Endpoint	Descripción
Pacientes	POST	`/api/pacientes`	Crear paciente
Pacientes	GET	`/api/pacientes`	Listar pacientes (búsqueda opcional con `?q=`)
Pacientes	GET	`/api/pacientes/{id}`	Consultar paciente
Pacientes	PUT	`/api/pacientes/{id}`	Actualizar paciente
Pacientes	PATCH	`/api/pacientes/{id}/habitacion`	Cambiar solo la habitación del paciente
Pacientes	DELETE	`/api/pacientes/{id}`	Dar de baja al paciente (baja lógica: se conserva con su historial)

3.3 TrabajadorController

Microservicio	Método	Endpoint	Descripción
Trabajadores	POST	`/api/trabajadores`	Crear trabajador (doctor o enfermero)
Trabajadores	GET	`/api/trabajadores`	Listar trabajadores (búsqueda opcional con `?q=`)
Trabajadores	GET	`/api/trabajadores/{id}`	Consultar trabajador
Trabajadores	PUT	`/api/trabajadores/{id}`	Actualizar trabajador
Trabajadores	DELETE	`/api/trabajadores/{id}`	Eliminar trabajador (también desactiva su usuario)

3.4 HistorialClinicoController

Microservicio	Método	Endpoint	Descripción
Historial clínico	POST	`/api/pacientes/{id}/historial`	Crear un registro clínico del paciente (cabecera opcional `Idempotency-Key` para no duplicarlo)
Historial clínico	GET	`/api/pacientes/{id}/historial`	Listar el historial de un paciente
Historial clínico	GET	`/api/historial`	Listar el historial general (`?filtro=paciente|autor&q=`)

3.5 UsuarioController

Microservicio	Método	Endpoint	Descripción
Usuarios	POST	`/api/usuarios`	Crear usuario
Usuarios	GET	`/api/usuarios`	Listar usuarios (búsqueda opcional con `?q=`)
Usuarios	GET	`/api/usuarios/{id}`	Consultar usuario
Usuarios	PATCH	`/api/usuarios/{id}/rol`	Cambiar el rol del usuario
Usuarios	PATCH	`/api/usuarios/{id}/desactivar`	Desactivar usuario (baja lógica)
Usuarios	PATCH	`/api/usuarios/{id}/activar`	Reactivar usuario
Usuarios	PATCH	`/api/usuarios/{id}/password`	Restablecer la contraseña de un usuario

3.6 MedicamentoController (puerta de entrada a MedicamentosService)

Estos endpoints validan el token y el rol del usuario y reenvían la petición a MedicamentosService con `MedicamentosClient`.

Microservicio	Método	Endpoint	Descripción
Medicamentos	POST	`/api/medicamentos`	Crear medicamento
Medicamentos	GET	`/api/medicamentos`	Listar medicamentos (búsqueda opcional con `?q=`)
Medicamentos	GET	`/api/medicamentos/{id}`	Consultar medicamento
Medicamentos	PUT	`/api/medicamentos/{id}`	Actualizar medicamento
Medicamentos	DELETE	`/api/medicamentos/{id}`	Eliminar medicamento
Medicamentos	POST	`/api/medicamentos/{id}/entradas`	Registrar entrada de stock
Medicamentos	POST	`/api/medicamentos/{id}/salidas`	Registrar salida de stock
Medicamentos	GET	`/api/medicamentos/stock-bajo`	Listar medicamentos con stock bajo
Medicamentos	GET	`/api/medicamentos/por-vencer`	Listar medicamentos por vencer (`?dias=`)
Medicamentos	GET	`/api/medicamentos/vencidos`	Listar medicamentos vencidos

4. Endpoints de MedicamentosService (puerto 8081)

4.1 MedicamentoController

Solo acepta peticiones de PachocloSystem con la cabecera `X-Api-Key`.

Microservicio	Método	Endpoint	Descripción
MedicamentosService	POST	`/api/medicamentos`	Crear medicamento con su stock inicial
MedicamentosService	GET	`/api/medicamentos`	Listar medicamentos (búsqueda opcional con `?q=`)
MedicamentosService	GET	`/api/medicamentos/{id}`	Consultar medicamento
MedicamentosService	PUT	`/api/medicamentos/{id}`	Actualizar medicamento
MedicamentosService	DELETE	`/api/medicamentos/{id}`	Eliminar medicamento
MedicamentosService	POST	`/api/medicamentos/{id}/entradas`	Sumar unidades al stock
MedicamentosService	POST	`/api/medicamentos/{id}/salidas`	Descontar unidades del stock (cabecera opcional `Idempotency-Key` para no descontar dos veces)
MedicamentosService	GET	`/api/medicamentos/stock-bajo`	Listar medicamentos con stock bajo
MedicamentosService	GET	`/api/medicamentos/por-vencer`	Listar medicamentos que vencen en los próximos `?dias=` (30 por defecto)
MedicamentosService	GET	`/api/medicamentos/vencidos`	Listar medicamentos vencidos

5. Decisiones de diseño

- **Historial clínico sin PUT ni DELETE:** un registro clínico es un documento médico y no se modifica ni se borra después de creado, para conservar la trazabilidad.
- **Usuarios sin DELETE:** los usuarios se desactivan con `PATCH /api/usuarios/{id}/desactivar` en lugar de borrarse, para conservar la cuenta y poder reactivarla con `PATCH /api/usuarios/{id}/activar`. El sistema impide desactivar al último administrador activo.
- **Baja lógica de pacientes:** `DELETE /api/pacientes/{id}` marca al paciente como inactivo en lugar de borrar la fila, para no perder su historial clínico. Desde ese momento la API lo trata como inexistente (404).
- **PATCH para cambios parciales:** la habitación del paciente, el rol, el estado y la contraseña de un usuario se cambian con PATCH porque solo modifican un campo; PUT se reserva para editar el recurso completo.
- **Entradas y salidas de stock con POST:** cada movimiento es una operación nueva sobre el inventario, por eso se crea con POST sobre `/{id}/entradas` y `/{id}/salidas`.